The Journal of Systems and Software 227 (2025) 112457

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

A comparative analysis of ML techniques for bug report classification ☆

Muhammad Laiq [a,*], Nauman bin Ali [a], Jürgen Börstler [a], Emelie Engström [b]

[a] Blekinge Institute of Technology, Department of Software Engineering, SE-371 79, Karlskrona, Sweden
[b] Lund University, Department of Software Engineering, SE-221 00, Lund, Sweden

ARTICLE INFO

Dataset link: https://github.com/laiqujan/ML-techniques-for-bug-report-classification/

Keywords:
Software maintenance
Issue classification
Bug report classification
Natural language processing
BERT
RoBERTa
Large language models
Automated machine learning
AutoML
Software analytics

ABSTRACT

Several studies have evaluated various ML techniques and found promising results in classifying bug reports. However, these studies have used different evaluation designs, making it difficult to compare their results. Furthermore, they have focused primarily on accuracy and did not consider other potentially relevant factors such as generalizability, explainability, and maintenance cost. These two aspects make it difficult for practitioners and researchers to choose an appropriate ML technique for a given context. Therefore, we compare promising ML techniques against practitioners' concerns using evaluation criteria that go beyond accuracy. Based on an existing framework for adopting ML techniques, we developed an evaluation framework for ML techniques for bug report classification. We used this framework to compare nine ML techniques on three datasets. The results enable a tradeoff analysis between various promising ML techniques. The results show that an ML technique with the highest predictive accuracy might not be the most suitable technique for some contexts. The overall approach presented in the paper supports making informed decisions when choosing ML techniques. It is not locked to the specific techniques, datasets, or factors we have selected here, and others could easily use and adapt it for additional techniques or concerns.

Editor's note: Open Science material was validated by the Journal of Systems and Software Open Science Board.

1. Introduction

Software system users submit bug reports describing deviations in system behavior from their expectations. Given the large number of daily submissions of bug reports, it is useful to be able to group them based on different criteria to support various maintenance tasks. In this study, we focus on a broad classification of bug reports: (a) bug reports that will require correcting an error in code, i.e., valid bug reports, and (b) all other bug reports, i.e., invalid bug reports. Such classification[1] enables routing only bug reports classified as valid to developers, saving the time they would otherwise spend on understanding, reproducing, and debugging bug reports that will turn out not to be code-related. Several studies have evaluated the use of ML-based techniques to classify bug reports (e.g. Zanetti et al., 2013; Fan et al., 2018; He et al., 2020; Kallis et al., 2019; Izadi, 2022).

The evaluation research has found promising results in automatically classifying bug reports (e.g., Fan et al., 2018; Laiq, 2023). The high accuracy of classification, as found in the assessments, is a necessary step toward encouraging the use of these techniques in practice (Ali, 2016).

However, there remains a gap in the body of knowledge in two areas. *First,* there is a lack of evaluations considering other aspects than accuracy. Other factors of concern for practical adoption are, for example, maintenance cost, scalability, explainability, and adaptability (Rana et al., 2014b; Paleyes et al., 2022). These factors do not always complement each other and may require a trade-off. For example, technique A may be more accurate than technique B, but technique A is more complex and scores poorly in terms of explainability. *Second,* existing evaluations are not easily comparable as they, in many cases, use different evaluation designs. For example, technique A may no longer be more accurate than technique B when different metrics or evaluation approaches for training and validation are used.

This study addresses these two gaps to help practitioners or researchers make informed decisions when selecting ML techniques for bug report classification tasks. We comprehensively evaluate the leading ML techniques for bug report classification (i.e., classical ML and BERT-based techniques and AutoML) with respect to factors that capture practitioners' concerns when adopting ML techniques in the industry. The contributions of this study are as follows.

☆ Editor: Alexander Serebrenik.

* Corresponding author.
E-mail addresses: muhammad.laiq@bth.se (M. Laiq), nauman.ali@bth.se (N.b. Ali), jurgen.borstler@bth.se (J. Börstler), emelie.engstrom@cs.lth.se (E. Engström).

[1] However, there may be bug reports categorized as valid that the developers find non-reproducible. They may later be resolved as invalid.

https://doi.org/10.1016/j.jss.2025.112457
Received 7 August 2024; Received in revised form 18 February 2025; Accepted 7 April 2025
Available online 21 April 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

- We adapted an existing framework for adopting ML techniques (Rana et al., 2014b) to evaluate ML techniques for bug report classification. The adapted framework considers several additional factors besides prediction accuracy, e.g., generalizability, explainability, and maintenance of the model. We used this evaluation framework to assess nine ML techniques on three datasets.
- We used data from both open-source and closed-source contexts to improve the generalizability of the results.
- We proposed and evaluated the use of AutoML for bug report classification. AutoML (Hutter et al., 2019; Feurer et al., 2015) enables automatically selecting suitable models, hyperparameters, and data preprocessing methods. It can, therefore, simplify the adoption of ML techniques for bug report classification in practice.
- We provide a decision matrix that visualizes tradeoffs between several promising ML techniques for bug report classification. This will support making informed decisions when selecting an appropriate ML technique for bug report classification.
- The overall approach used in this study is adaptable and can be readily applied or expanded by others to include additional techniques or adoption factors.

The remainder of the paper is organized as follows. Section 2 describes the related work. Section 3 presents the research method of the study. Section 4 presents the results and analysis. Section 5 discusses the findings of the study. Section 6 describes the validity threats of the study and Section 7 concludes the paper.

2. Related work

This section presents related work on the adoption of ML techniques in software development practice and the classification of bug reports.

2.1. Adoption of ML techniques in software development practice

Several studies have focused on adopting ML techniques in practice to assist practitioners in software engineering tasks, such as bug assignment and prediction (Rana et al., 2014a; Aktas and Yilmaz, 2020; Oliveira et al., 2021; Borg et al., 2022). However, to our knowledge, no study has focused on assisting the adoption decision of ML techniques for bug report classification against the adoption concerns of practitioners.

Paleyes et al. (2022) reviewed experience reports and case studies on industrial adoption of ML techniques and identified several challenges associated with the adoption of ML techniques in practice, such as the explainability of ML techniques and their predictions, computational cost, integration, and updating the ML model to handle concept drift. Rana et al. (2014b) also identified similar challenges/concerns of practitioners about adopting ML techniques in the context of bug prediction.

In this study, we utilize Rana et al.'s (Rana et al., 2014b) framework for the adoption of ML techniques in practice to assess the ML techniques for bug report classification.

2.2. Bug report classification

Several studies have focused on bug report classification in the literature, e.g., Pandey et al. (2017), Qin and Sun (2018), Otoom et al. (2019), Li et al. (2022). A workshop section (Kallis et al., 2023a) is also dedicated to the topic. This section presents the existing work on the topic, including the work based on the workshop and the work identified on the topic based on our previous work (Laiq et al., 2022).

Table 1 presents an overview of the identified related work on classifying bug reports, grouped by the techniques used for classification and the features, evaluation strategies, and evaluation metrics used to assess the used techniques. All of these studies have been conducted in an open-source context except for Laiq et al. (2022), which was conducted in a closed-source context.

In terms of techniques, classical ML and BERT-based techniques are predominant. We find that Naive Bayes, Logistic Regression, Support Vector Machines, and Random Forest are commonly used. A few studies (He et al., 2020; Li et al., 2022; Qin and Sun, 2018) have also used neural network-based techniques.

Various features, such as the title, summary, and description of a bug report, have been used to train the techniques to classify bug reports. Most studies only use the title and description of the bug report.

Among the evaluation strategies and metrics used in these studies, "test on a fixed set" and "F1 measure", respectively, are used most commonly. The variety in features, evaluation strategies, and metrics used makes it challenging to select a suitable technique for a specific context because the results of the proposed techniques are not comparable. These differences may also explain seemingly different results in some studies. For example, Fan et al. (2018), Otoom et al. (2019), and Pandey et al. (2017) reported different results regarding the use of SVM for the classification of bug reports.

Similarly, Pingclasai et al. (2013) and Terdchanakul et al. (2017) reported varying results about using Logistic Regression for bug report classification. In addition, Fan et al. (2018) and Pingclasai et al. (2013) reported different results regarding the use of Naive Bayes for bug report classification.

Different results in accuracy for the same technique for the same task may be explained by the use of different setups in these studies. (a) *evaluation strategy:* Fan et al. (2018) used "time split with 10-fold CV", Otoom et al. (2019) used "5-fold CV", and Pandey et al. (2017) used "test on a fixed set". (b) *evaluation metrics:* Fan et al. used "AUC and F1 measure", Otoom et al. (2019) used "accuracy", and Pandey et al. (2017) used "accuracy and F1 measure". (c) *features:* Fan et al. (2018) used "summary, description, text completeness, readability and complexity of a bug report and submitter experience and features based collaboration graph", Otoom et al. (2019) used "description and title of a bug report", and Pandey et al. (2017) used "summary of a bug report".

Furthermore, the main focus of these studies is on the accuracy of the proposed techniques for bug report classification.

The possible explanation for not considering adoption factors beyond prediction accuracy in these studies could be due to the objectives of these studies, i.e., they aim to evaluate and compare the prediction performance of existing and proposed techniques. Another potential reason could be that these studies focus mainly on mining the OSS data and do not involve practitioners. However, when we try to transfer the techniques to practitioners so that they may use them during software maintenance, these additional factors become important.

The lack of evaluations that consider factors other than accuracy and the use of different evaluation designs makes it difficult for practitioners and researchers to choose an appropriate ML technique for a given context. For example, it is difficult to decide which technique is best in terms of prediction performance since these studies have used different evaluation designs, as reported above. Furthermore, making an informed decision when selecting an ML technique for a given context requires considering additional factors such as explainability and computational cost. For example, while Technique A may be more accurate than Technique B, it could score poorly in terms of explainability. The existing work on adopting ML techniques in practice also highlights that when practitioners select ML techniques for adoption, they consider several other factors beyond prediction accuracy (Rana et al., 2014b; Paleyes et al., 2022). For example, generalizability, explainability, and scalability of ML techniques.

Therefore, there is a need to evaluate these techniques using the same design with more extensive criteria to facilitate their adoption decision in a particular context. Thus, in this study, we aim to assist the adoption decision by evaluating several ML techniques for bug report classification against the concerns of practitioners using a consistent evaluation design.

Table 1
Techniques, features, evaluation strategies, and evaluation metrics used in previous work (25 studies). Except for the evaluation strategies, column *Count* does not add up to 25, since many studies used multiple techniques, features, and evaluation metrics.

Techniques used in previous work	Studies	**Count**
Naive Bayes	Antoniol et al. (2008), Pingclasai et al. (2013), Zhou et al. (2016), Pandey et al. (2017), Otoom et al. (2019), Herbold et al. (2020)	6
Logistic Regression	Antoniol et al. (2008), Pingclasai et al. (2013), Terdchanakul et al. (2017), Izadi (2022), Laiq et al. (2022)	5
Support Vector Machines	Zanetti et al. (2013), Pandey et al. (2017), Fan et al. (2018), Otoom et al. (2019), Laiq et al. (2022)	5
Random Forest	Pandey et al. (2017), Fan et al. (2018), Terdchanakul et al. (2017), Otoom et al. (2019), Herbold et al. (2020)	5
Decision Tree	Antoniol et al. (2008), Pingclasai et al. (2013), Pandey et al. (2017), Laiq et al. (2022)	4
RoBERTa	Izadi (2022), Colavito et al. (2022), Kallis et al. (2023b)	3
BERT-based Model	Trautsch and Herbold (2022), Siddiq and Santos (2022), Bharadwaj and Kadam (2022)	3
Fasttext	Kallis et al. (2019), Herbold et al. (2020), Kallis et al. (2023b)	3
Long Short-term Memory	Qin and Sun (2018), Li et al. (2022)	2
K-nearest Neighbors	Pandey et al. (2017), Laiq et al. (2022)	2
Stochastic Gradient Descent-based Classifier	Laiq (2023)	1
Fuzzy Logic	Chawla and Singh (2015)	1
Bayesian Networks	Zhou et al. (2016)	1
Linear Discriminant Analysis	Pandey et al. (2017)	1
Classification Associations Rule Mining	Zolkeply and Shao (2019)	1
Convolutional Neural Network	He et al. (2020)	1
Multilayer Perceptron	Colavito et al. (2022)	1
XLNet	Bharadwaj and Kadam (2022)	1
Few-shot Learning and Sentence-BERT-based Model	Colavito et al. (2023)	1
Feed-Forward Neural Network with OpenAI's Text-embedding-ada-002 Model	Du et al. (2024)	1
Features used in previous work	Studies	**Count**
Description of a bug report	Antoniol et al. (2008), Pingclasai et al. (2013), Fan et al. (2018), Qin and Sun (2018), Terdchanakul et al. (2017), Kallis et al. (2019), Otoom et al. (2019), Zolkeply and Shao (2019), He et al. (2020), Herbold et al. (2020), Izadi (2022), Li et al. (2022), Laiq et al. (2022), Trautsch and Herbold (2022), Siddiq and Santos (2022), Colavito et al. (2022), Bharadwaj and Kadam (2022), Kallis et al. (2023b), Colavito et al. (2023), Laiq (2023), Du et al. (2024)	21
Title/heading of a bug report	Antoniol et al. (2008), Chawla and Singh (2015), Terdchanakul et al. (2017), Kallis et al. (2019), Otoom et al. (2019), Herbold et al. (2020), Izadi (2022), Li et al. (2022), Laiq et al. (2022), Trautsch and Herbold (2022), Siddiq and Santos (2022), Colavito et al. (2022), Bharadwaj and Kadam (2022), Kallis et al. (2023b), Colavito et al. (2023), Laiq (2023), Du et al. (2024)	17
Summary of a bug report	Zhou et al. (2016), Pandey et al. (2017), Qin and Sun (2018), Fan et al. (2018), He et al. (2020)	5
Priority of a bug report	Zhou et al. (2016), Laiq et al. (2022)	2
Features based on collaboration graphs of bug submitters, e.g., the clusters of highly connected people	Zanetti et al. (2013), Fan et al. (2018)	2
Submitter experience (e.g., total submitted bug reports)	Fan et al. (2018), Laiq et al. (2022)	2
Discussion text of a bug report	Antoniol et al. (2008), Du et al. (2024)	2
Severity, component, assignee, reporter, operating system, and reproducibility of a bug report	Zhou et al. (2016)	1

(*continued on next page*)

Table 1 (*continued*).

Techniques used in previous work	Studies	**Count**
Text completeness, readability and complexity of a bug report	Fan et al. (2018)	1
Evaluation strategies used in previous work	Studies	**Count**
Test on a fixed set	Zanetti et al. (2013), Chawla and Singh (2015), Pandey et al. (2017), Zolkeply and Shao (2019), Izadi (2022), Trautsch and Herbold (2022), Siddiq and Santos (2022), Colavito et al. (2022), Bharadwaj and Kadam (2022), Kallis et al. (2023b), Colavito et al. (2023), Laiq (2023)	12
10-fold CV	Antoniol et al. (2008), Pingclasai et al. (2013), Zhou et al. (2016), Kallis et al. (2019), Li et al. (2022)	5
Time split with 10-fold CV	Fan et al. (2018), Terdchanakul et al. (2017), Laiq et al. (2022)	3
Time split with test on a fixed set	Qin and Sun (2018), He et al. (2020)	2
5-fold CV	Otoom et al. (2019), Du et al. (2024)	2
Leave-one-project-out CV	Herbold et al. (2020)	1
Evaluation metrics (underlying) used in previous work	Studies	**Count**
F1 measure	Pingclasai et al. (2013), Zanetti et al. (2013), Chawla and Singh (2015), Zhou et al. (2016), Pandey et al. (2017), Fan et al. (2018), Qin and Sun (2018), Terdchanakul et al. (2017), Kallis et al. (2019), He et al. (2020), Herbold et al. (2020), Li et al. (2022), Du et al. (2024)	13
Micro average (F1, Precision, and Recall)	Izadi (2022), Trautsch and Herbold (2022), Siddiq and Santos (2022), Colavito et al. (2022), Bharadwaj and Kadam (2022), Kallis et al. (2023b), Colavito et al. (2023), Laiq (2023)	8
Precision	Antoniol et al. (2008), Zanetti et al. (2013), Chawla and Singh (2015), Zhou et al. (2016), Kallis et al. (2019), Herbold et al. (2020), Li et al. (2022), Du et al. (2024)	8
Recall	Antoniol et al. (2008), Zanetti et al. (2013), Chawla and Singh (2015), Zhou et al. (2016), Kallis et al. (2019), Herbold et al. (2020), Li et al. (2022), Du et al. (2024)	8
Accuracy	Chawla and Singh (2015), Pandey et al. (2017), Otoom et al. (2019), Zolkeply and Shao (2019), Du et al. (2024)	5
AUC	Fan et al. (2018), He et al. (2020), Laiq et al. (2022)	3
MCC	Laiq et al. (2022)	1

3. Research method

This study aims to comprehensively evaluate ML techniques for bug report classification. To achieve this goal, we posed the following overarching research question: **How do ML techniques perform for bug report classification?** To answer the research question, we conduct comparative experiments (Alpaydin, 2020; Wohlin et al., 2012).

The following subsections describe and justify the main decisions taken in the design of the experiments, including the selection of ML techniques, the selection of evaluation criteria, the selection of datasets, and the choice of the evaluation approach.

3.1. Selection of ML techniques

As reported in Section 2.2, several ML techniques have been proposed in the literature for bug report classification. These techniques have been proposed to classify bug reports into the following two broad categories. The first category includes issues that require an update to the current system behavior in the form of a code fix. The second category includes issues that do not require an update to the current system behavior in the form of a code fix. The second category may include sub-categories, such as misunderstandings of the system's functionality, faulty documentation, and new requirements. In the following, we describe our choice of techniques to enable trade-offs and support practitioners in making informed decisions when choosing an appropriate technique for bug report classification.

Classical ML techniques. We chose commonly proposed classical (traditional) ML techniques in the literature for bug report classification to assess their performance against the adoption concerns of practitioners: Naive Bayes (NB), Logistic Regression (LR), Support Vector Machines (SVM), Random Forest (RF), Decision Tree (DT), K-Nearest Neighbors (KNN), and Stochastic Gradient Descent (SGD)-based classifier.

We opted for classical ML techniques due to their effectiveness in accurately classifying bug reports. The findings from the literature (e.g., Pandey et al., 2017; He et al., 2020; Laiq, 2023) indicate that classical ML techniques can accurately classify bug reports and can achieve results comparable to complex techniques (e.g., deep learning and BERT-based) while offering additional advantages such as less computational overhead (Kallis et al., 2023a). For example, the findings of Li et al. (2022) have shown that classical ML techniques such as LR proposed by Pandey et al. (2017) can achieve results comparable to complex techniques such as long short-term memory, proposed by Li et al. (2022) and Qin and Sun (2018). Likewise, He et al. (2020) reported that their deep-learning-based technique could not outperform the classical ML techniques (i.e., SVM and RF) proposed by Fan et al. (2018) on all datasets. Similarly, Laiq (2023) showed that a simple classical ML technique (i.e., an SGD-based classifier equivalent to SVM) can achieve comparable results to BERT-based techniques and outperform Fasttext (Joulin et al., 2016) in classifying bug reports.

BERT-based model. In addition to the classical ML techniques mentioned above, this study also considers the state-of-the-art BERT-based model (i.e., RoBERTa) for bug report classification (Kallis et al., 2023a). We chose to fine-tune the RoBERTa model instead of just using it for text embeddings. This method involves adjusting the model's parameters to fit specific tasks better, resulting in better outcomes than using a pre-trained large language model (e.g., RoBERTa) for text embeddings with a separate classifier, e.g., logistic regression (Devlin et al., 2018).

AutoML. Despite the promising results of the above techniques, setting up these techniques requires manual effort and expertise to select a suitable model, data preprocessing method, and hyperparameters.

Research (Hutter et al., 2019; Feurer et al., 2015) has shown that AutoML has the potential to automate the selection of suitable models, data preprocessing methods, and hyperparameters. Thus, if AutoML can produce comparable accuracy it can be a candidate for adoption in practice. Thus, in addition to the above-mentioned techniques that require manual training and optimization, we propose the use of AutoML for bug report classification. To the best of our knowledge, our study is the first to propose and evaluate the use of AutoML for bug report classification.

The term AutoML stands for "automated machine learning". For a given task and dataset where ML is to be used, AutoML provides a mechanism to automate the selection of suitable ML techniques, data preprocessing methods, and setting up hyperparameters (Hutter et al., 2019; Feurer et al., 2015).

AutoML's popularity has increased in different domains over the past years and is also gaining attention in software engineering (Calefato et al., 2023). For example, Tanaka et al. (2019) used AutoML for defect prediction and found that AutoML can achieve results similar to those of the best-performing ML techniques in the literature for defect prediction, i.e., RF. Similarly, Calefato et al. (2023) used AutoML for sentiment analysis and found that AutoML-generated models can outperform the models trained and optimized by researchers. In addition, they reported that AutoML-generated models can achieve performance close to that of time-consuming and complex techniques based on fine-tuning pre-trained transformers-based models, for example., BERT (Devlin et al., 2018).

3.2. Evaluation criteria for ML techniques

To evaluate the ML techniques for classifying bug reports against the adoption concerns of practitioners, we adapted Rana et al.'s framework (Rana et al., 2014b). Their framework consists of three dimensions, which we briefly describe below:

1. **Characteristics of ML techniques:** It includes 14 attributes relevant to an ML technique that practitioners consider relevant when deciding whether to adopt a technique. For example, a technique's accuracy, scalability, or generalizability.
2. **Organizational characteristics:** It includes 11 attributes relevant to an organization that may affect the acceptance of a proposed ML technique, for example, the satisfaction of an organization with existing systems and competency with ML.
3. **External environment:** It includes two attributes relevant to an external environment of an organization (i.e., the adoption of an ML technique by competitors or other industries) that may affect the adoption of a proposed ML technique.

Among the three dimensions of the framework (Rana et al., 2014b) listed above, the assessment of the last two dimensions requires input specific to a context, i.e., each adopting organization has to assess the attributes of these two dimensions. For example, the familiarity and competence of engineers with ML techniques would need to be assessed. However, the attributes related to the first dimension (i.e., characteristics of ML techniques) do not have such dependency and can largely be assessed independently of a specific context. Thus, by focusing on assessing the characteristics of ML techniques, this study provides the necessary information that, in conjunction with context-specific dimensions, will allow for an informed selection of ML techniques for bug report classification.

The first dimension (i.e., characteristics of ML techniques) includes 14 attributes. Of those 14 attributes, we chose 8 attributes that we consider related to the characteristics of ML techniques. Two of the 14 attributes were merged with other attributes, e.g., 'Low generalizability of ML technique' was merged with 'Generalizability of ML technique.' Four of the 14 attributes were excluded for the following reasons: (a) Two were excluded because they require external input and are not related to the characteristics of the ML techniques. (b) An attribute requires an external input. (c) An attribute is used to identify patterns from data instead of making predictions using an ML technique.

Table 2 briefly describes each attribute and our judgment on whether the attribute is relevant to the "characteristics of ML techniques" and can be assessed without external input. In Table 3, we describe the measurement scales of the attributes assessed in this study.

3.3. Selection of datasets

To evaluate the selected ML techniques, we chose three datasets of bug reports, D1–D3, see Table 4. Two of these datasets are from open-source context, provided by Herzig et al. (2013) and Kallis et al. (2023a). Several studies discussed in the related work (see Section 2.2) have used these two datasets as a benchmark to evaluate ML techniques for bug report classification.

The dataset by Herzig et al. (2013) includes bug reports from 5 open-source projects. Three out of five projects, Lucene-Java, Jackrabbit, and HTTPClient, use Jira as their bug tracker. The other two projects, Rhino and Tomcat5, utilize Bugzilla for bug tracking.

The dataset by Kallis et al. (2023a) includes bug reports from open-source GitHub projects.[2]

The dataset D1 is from a product of a large telecommunication company (called CSS in the following), a closed-source context. Their context can be categorized as large-scale software-intensive system development. D1 includes bug reports from 2018–2023. Compared to open-source datasets (D2 and D3), D1 has a smaller percentage of invalid bug reports (see Table 5 for the percentage of invalid bug reports over the years). This disparity may be attributed to the rigorous processes that are followed for bug writing and reviewing in a proprietary context and the discrepancy in the skills and motivation of the bug submitters (Bachmann and Bernstein, 2009).

We chose more than one dataset and data from both OSS and closed-source contexts to test which techniques perform reliably on different datasets. These datasets include not only bug reports from different contexts (i.e., OSS and CSS) but also from different projects, e.g., D2 includes data from five different projects. As shown in Table 4, the datasets differ in size. Furthermore, the distribution of classes varies significantly, with D1 having a 13:87 ratio, D2 a 35:65 ratio, and D3 being closer to equal at 48:52. These variations in context, projects, size, and class distribution will further help in assessing the reliability in performance of the chosen techniques.

Similar to most of the previous work (see Section 2), in this study, we use the title and description of bug reports for bug report classification, i.e., from all three datasets, we extract and use only textual features (titles and descriptions) of each bug report.

3.4. Experiment design

The following subsections describe how our experiments were designed and conducted. We followed the guidelines by Alpaydin (2020) for designing and conducting ML experiments and the general guidelines for experimentation in software engineering by Wohlin et al. (2012).

3.4.1. Variables and factors

Dependent variables: The following four attributes of the selected ML techniques: prediction accuracy, computational cost, generalizability, and scalability. The measurement scales of these attributes are shown in the first four rows of Table 3.

Independent variables: (a) the selected techniques: NB, LR, SVM, RF, DT, KNN, SGD, RoBERTa, AutoML-30, AutoML-60, and AutoML-90 (see Section 3.1), and (b) the three datasets: D1, D2, and D3 (see Section 3.3).

[2] https://github.com/nlbse2023/issue-report-classification/blob/main/1-Dataset.ipynb

Table 2
Attributes describing characteristics of ML techniques (based on Rana et al. (2014b)). An "N/a" in column *Attribute* refers to attributes from Rana et al. that we have not included or that we have merged with another attribute.

Attribute	Original name	Description/rationale
Accuracy of ML technique	Accuracy in predicting	This attribute is used for the prediction accuracy of an ML technique. The attribute can be assessed using several evaluation metrics for ML techniques, such as AUC, accuracy, and F1-score.
Computational cost	Low cost of obtaining results	The estimated amount of resources (cost) an ML technique consumes to produce results. In this study, we only consider the following types of costs: (a) training and testing time of an ML technique and (b) memory usage.
Generalizability of ML technique	Adaptability to different datasets	Ability of an ML technique to make predictions with comparable accuracy on different datasets (projects) for the same task (Rana et al., 2014b). A particular company can decide on this attribute based on a cut-off value for accuracy.
Scalability of ML technique	Ability to handle large data	The ability of an ML technique to handle a large amount of data. Zhang and Tsai (2003) argue that ML techniques can handle large amounts of data and, in fact, are expected to improve their performance as more data is available. In our opinion, getting comparable or better accuracy is intuitive by increasing the amount of data. However, other important attributes should be assessed to evaluate the scalability of an ML technique, such as the time to train and produce results and the required RAM for large datasets.
Learning curve	Steep learning curve	The knowledge required to get optimal results using an ML technique (Rana et al., 2014b). For example, selecting the set of attributes (input) for the ML technique to produce optimal results, e.g., the use of appropriate hyperparameters.
Explainability	This attribute does not exist in Rana et al.	The explainability for an ML technique can be achieved at global and local levels (Molnar, 2018), i.e., (a) global explainability: explaining the overall working of an ML model and (b) local explainability: explaining the prediction results of an individual instance. Practitioners consider this factor a must-have when deciding to adopt an ML technique in practice (Bhatt et al., 2020).
Maintenance of the model	Automation of pattern discovery	Ability of an ML technique to recognize and handle new patterns in data to maintain its performance over time.
Availability of open-source tools	Availability of open-source tools	Availability of open-source tools to implement ML techniques/algorithms.
Technical support	Consulting services	The availability of technical support regarding the ML techniques (Rana et al., 2014b), e.g., tutorial about implementing ML techniques or interpreting their results.
N/a	Low generalizability of ML technique	Merged with "Generalizability of ML technique."
N/a	Inability to recognize new patterns	Merged with "maintenance of the model."
N/a	Compatibility with existing systems	The compatibility of the ML tools or libraries used to implement ML techniques with existing systems or pipelines at a particular organization. This attribute requires external input, i.e., an organization can only assess the compatibility of an ML-based tool with existing systems.
N/a	Ability to generate new insights	Ability of an ML technique to identify new patterns in data and automatically provide new insights. Authors of the original framework (Rana et al., 2014b) suggest that new insights can be generated using ML techniques, such as Bayesian Networks. This study focuses on the use of ML techniques to classify bug reports, and this attribute is related to the evaluation of the ML technique's ability to generate new insights, such as identifying patterns or trends from bug reports using clustering or topic modeling techniques.
N/a	High cost of misprediction	This attribute assesses the impact/cost of misprediction. In our opinion, this attribute is not related to the characteristics of an ML technique, and it also requires external input, i.e., an organization can only estimate how much mispredictions will cost/impact.
N/a	The need for high coverage of factors/variables	In our opinion, this attribute is not related to the characteristics of an ML technique, and it also requires external input, i.e., an organization can list prioritized or essential adoption attributes that should be covered to adopt an ML technique.

Fixed/Controlled variables: Initial data preprocessing approach, evaluation approach, and used features.

Factors: 2 factors: ML technique and dataset.

3.4.2. Evaluation approach

Similar to previous work (see Section 2.2), we evaluated our models using a 10-fold cross-validation strategy. To perform 10-fold cross-validation, we divided our dataset into 10 equal folds. We then trained the models on nine folds each time and tested them on a single fold. This process was repeated ten times. Thus, different test sets were used each time. Finally, we averaged the results over 10 folds for each ML technique. Furthermore, we do not perform cross-evaluations between datasets as our datasets (D1, D2, and D3) are from different contexts.

3.4.3. Classification categories

Bug reports can be categorized based on different criteria, such as the severity, type, and validity of the underlying issue. Each classification scheme has significant implications for software development. For example, accurately classifying bug reports based on validity can help prioritize issues and effectively assign relevant resources.

Concerning their validity, bug reports are broadly classified into two categories depending on the issues they describe (Zanetti et al., 2013; Fan et al., 2018). The first category includes issues that require an update to the current system behavior in the form of a code fix. These bug reports are also known as valid bug reports. The second category includes issues that do not require an update to the current system behavior in the form of a code fix. These bug reports are also known as invalid bug reports. The bug reports into the second category may include reports submitted due to, e.g., misunderstandings of the system's functionality, faulty documentation, and non-existent requirements.

Early classification (as soon as a bug report is filed in a bug tracking system or before it is assigned to a relevant developer/team) of bug reports based on their validity can help companies in the bug management process to assign resources to bug reports effectively, e.g., prioritizing valid bug reports and deferring invalid bug reports for

Table 3
Assessment methods for the attributes describing characteristics of ML techniques.

Attribute	Assessment methods
Accuracy of ML technique	AUC value, see further details in Section 3.4.4 on the choice of AUC.
Computational cost	
(a) Training and testing time of an ML technique	Time in minutes.
(b) Memory usage	RAM and GPU in megabytes (MBs).
Generalizability of ML technique	A particular organization can decide on this attribute based on a cut-off value of AUC. In this study, to assess the generalizability of an ML technique, we measure the standard deviation (SD) and average AUC of each technique across the datasets.
Scalability of ML technique	Measured by analyzing the time, memory, and accuracy of ML techniques while considering the dataset size.
Learning curve	Subjective opinion about the knowledge required to use an ML technique, e.g., selecting relevant data preprocessing methods and hyper-parameters.
Explainability	
(a) Global explainability, i.e., explaining the ML model	Subjective opinion based on the existing literature and official documentation of the techniques used.
(b) Local explainability, i.e., explaining the results of a prediction	Subjective opinion based on the existing literature and official documentation of the techniques used.
Maintenance of the model	Subjective opinion on how the technique can handle the underlying changes (patterns in data) to maintain its performance over time.
Availability of open-source tools	Subjective information about open-source tools/libraries to implement ML techniques.
Technical support	Subjective information about available support in the form of tutorials, official documentation, or community support regarding ML techniques and frameworks/libraries to implement/use them for bug report classification.

Table 4
Bug reports data: D1 from CSS, D2 by Herzig et al. (2013), and D3 by Kallis et al. (2023a).

Id	Valid bug reports (%)	Invalid bug reports (%)	Total bug reports
D1	87%	13%	3300
D2	65%	35%	5591
D3	52.5%	47.5%	1418201

Table 5
Invalid bug reports percentage over the years for D1.

Year	Invalid bug reports (%)
2018	17.2872%
2019	12.1114%
2020	14.1791%
2021	11.7521%
2022	7.7586%
2023	9.4276%

the time being. Due to the importance of identifying valid bug reports early on in practice, in this study, we focus on using ML techniques to classify bug reports into the two categories mentioned above: valid and invalid.

3.4.4. Evaluation metrics

Herzig et al.'s (Herzig et al., 2013) dataset and the dataset from CSS are imbalanced. Thus, to evaluate the performance of the ML techniques on imbalanced data, we use a metric considered robust to data imbalance, i.e., Area Under the Curve (AUC) of receiver operator characteristic (ROC) (Huang and Ling, 2005) using the sklearn library (scikit-learn developers, 2024b). The AUC value is computed by plotting the ROC curve using the True Positive Rate (TPR) and False Positive Rate (FPR).

$$TPR = \frac{TP}{TP + FN} \tag{1}$$

$$FPR = \frac{FP}{FP + TN} \tag{2}$$

3.4.5. Data preprocessing

We apply the following standard data preprocessing to the titles and descriptions of bug reports on all three datasets (see Section 3.3) mainly using the Gensim library (Ř.ehůřek and Sojka, 2010):

1. Merge the **title** and **description** fields of each bug report.
2. Remove special characters, HTML tags, punctuations, numbers, consecutive whitespaces, and stopwords.
3. Convert labels of each bug report to numbers, i.e., *valid:1, invalid: 0.*
4. (a) Apply Tf-idf (scikit-learn developers, 2024) on the cleaned data and generate sparse matrices. Sparse matrices are only generated for the manually trained and optimized classical ML techniques using sklearn. For AutoML, we let Auto-sklearn decide which preprocessing method to apply to the text of bug reports.
   (b) For implementing RoBERTa, we perform additional standard data preprocessing, such as encoding. We used the script provided by Siddiq and Santos (2022).

Our choice of data preprocessing approaches is motivated by our aim of comparing the performance of classical ML techniques, AutoML, and BERT-based models. We only used the traditional approach of TF-IDF with classical ML techniques as it is a traditional approach and the most commonly used approach with these techniques. For the BERT-based model, we use a BERT-based tokenizer; for AutoML, we let AutoML decide which approach to use.

3.4.6. Implementation details

We used the sklearn (scikit-learn developers, 2024a) library for implementing the selected classical ML techniques (see Section 3.1). The best parameters for the techniques are determined by performing the grid search supported by sklearn. First, we tune the parameters of each of the selected classical ML techniques using the grid search, and then we use the best parameters to create a model.

Similar to Tanaka et al. (2019), in this study, we chose Auto-sklearn (Feurer et al., 2015) to assess the use of AutoML for bug report classification. Auto-sklearn uses sklearn (scikit-learn developers, 2024a) to implement classical ML techniques, including evaluation metrics, evaluation strategies, and preprocessing methods. To generate a model using Auto-sklearn, we use basic parameter settings, such as setting the evaluation strategy as 10-fold cross-validation and allocating time and memory. We performed our experiments using three different time slots, i.e., 30, 60, and 90 min, hereafter called AutoML-30, AutoML-60, and AutoML-90. We started with a 30 min time slot and then added 30 min to the remaining slots. However, we did not observe

Table 6
Evaluation of the selected ML techniques for bug report classification on three datasets of bug reports, i.e., D1 (bug reports data from CSS), D2 (Herzig et al.'s (Herzig et al., 2013) dataset), and D3 (10k randomly selected bug reports from Kallis et al.'s (Kallis et al., 2022) dataset).

Attributes	ML techniques										
	NB	LR	SVM	SGD	KNN	RF	DT	AutoML-30	AutoML-60	AutoML-90	RoBERTa
Accuracy (AUC) on three datasets											
D1	0.61	0.63	0.65	0.63	0.50	0.57	0.62	0.67	**0.68**	**0.68**	**0.69**
D2	0.81	0.81	0.80	0.80	0.73	0.72	0.71	0.86	**0.87**	**0.88**	**0.84**
D3	0.74	0.81	0.81	0.80	0.69	0.77	0.78	**	**0.85**	**0.86**	**0.85**
Computational cost for training the models in terms of training time in minutes on three datasets											
D1	0.04	3.17	17.56	0.09	0.31	12.55	0.95	30	60	**90**	**245.90**
D2	0.16	20.51	18.55	0.16	0.65	20.43	2.04	30	60	**90**	**171.00**
D3	0.14	15.79	26.63	0.33	3.43	**89.74**	8.55	30	60	**90**	**298.60**
Computational cost for training the models in terms of RAM usage in MBs on three datasets											
D1	205.74	205.21	210.15	203.06	209.64	202.56	209.83	520.37	**2576.74**	**2192.25**	**2513.30**
D2	197.57	191.18	208.86	191.30	196.72	232.23	190.24	**2039.09**	**2431.97**	**6557.09**	**2216.99**
D3	318.77	385.75	359.79	294.29	313.78	177.52	286.99	214.61	**1441.07**	**8675.49**	**2116.36**
Computational cost for training the models in terms of GPU memory usage in MBs on three datasets											
D1											**4000.00**
D2	Only CPU was used, and its memory consumption is reported in the previous row.										**8394.00**
D3											**8508.00**
Generalizability in terms of average AUC across datasets											
	0.72	0.75	0.75	0.74	0.64	0.69	0.70	0.51	**0.80**	**0.81**	**0.79**
Generalizability in terms of standard deviation of AUC across datasets											
	0.08	0.08	**0.07**	0.08	0.10	0.09	**0.07**	0.37	0.08	0.09	**0.07**

** No model was generated using the 30 min time slot for AutoML on dataset D3.

any notable changes in the performance. Thus, we stopped after the 90 min slot. Tanaka et al. (2019) also reported similar observations.

For BERT, we used the script[3] provided by Siddiq and Santos (2022). Unlike them, we use (fine-tune) RoBERTa instead of 'bert-base-uncased' as Kallis et al. (2023a) showed RoBERTa could achieve slightly better performance than the models based on 'bert-base-uncased' for bug report classification. In addition, to have a consistent evaluation of all the techniques assessed in this study, we also performed 10-fold cross-validation.

To perform our experiments using the classical ML techniques and AutoML, we used a machine equipped with a Windows 10 64-bit operating system, Intel (R) Core (TM) i7, and 16 GB RAM. The machine is used to perform experiments on three datasets of bug reports, i.e., D1 (bug reports data from CSS), D2 (Herzig et al.'s (Herzig et al., 2013) dataset), and D3 (10k randomly selected bug reports from Kallis et al.'s (Kallis et al., 2022) dataset). See details of datasets in Table 4.

To further analyze the scalability (one of the essential attributes of an ML technique that practitioners consider when deciding to adopt the ML technique, see details in Table 2) of AutoML and the selected classical ML techniques for bug report classification, we also applied AutoML and a few of the classical ML techniques with manual training and optimization on a complete dataset of bug reports by Kallis et al. (2023a), i.e., over 1.4 million bug reports. Since the dataset is quite large, we used a more powerful machine to conduct our experiments using AutoML than the one reported above for the datasets D1, D2, and D3. We used a machine equipped with Ubuntu 22.04 as an operating system, 135 GB RAM, NVIDIA GeForce RTX 2080 GPU, and 16 cores. The same machine was also used for implementing RoBERTa. The GPU was only utilized for RoBERTa.

4. Results and analysis

To answer the posed research question (i.e., **How do ML techniques perform for bug report classification?**), we evaluated the performance of several ML techniques (see Section 3.1 for details) against the adoption concerns of practitioners (see Table 2 for details). We applied the selected ML techniques on three datasets of bug reports, i.e., D1 (bug reports data from CSS), D2 (Herzig et al.'s (Herzig et al., 2013) dataset), and D3 (10k randomly selected bug reports from Kallis et al.'s (Kallis et al., 2022) dataset). See details of datasets in Table 4.

In the following subsections, we present the assessment results of ML techniques against the adoption concerns of practitioners for bug report classification.

4.1. Prediction accuracy

As shown in Table 6, AutoML-generated models performed better than manually trained and optimized ML models of classical ML techniques in classifying bug reports for all three datasets. Similarly, AutoML-generated models performed slightly better than the BERT-based model (i.e., RoBERTa). At best, using AutoML, we achieved an AUC of 0.68, 0.88, and 0.86 on D1, D2, and D3 datasets, respectively. On the other hand, using manual optimization on classical ML techniques, at best, we achieved an AUC of 0.65, 0.81, and 0.81 on D1, D2, and D3, respectively. Using RoBERTa, we achieved an AUC of 0.69, 0.84, and 0.85 on datasets D1, D2, and D3, respectively.

Takeaways:

1. AutoML, RoBERTa, and the manually trained and optimized classical ML techniques classify bug reports with AUC ranging from 0.69–0.88 across datasets.
2. Manually trained and optimized classical ML techniques achieve results comparable to complex techniques in classifying bug reports.
3. AutoML-generated models outperform manually trained and optimized models of classical ML techniques for bug report classification and achieve slightly higher AUC than RoBERTa.
4. Using only textual features, such as the title and description of bug reports, may not be sufficient to accurately classify bug reports in all contexts. For instance, on the dataset from CSS, we achieved an AUC of 0.69 at best. To achieve higher accuracy, additional features of bug reports should be considered, such as the experience of a bug report submitter. Further examples of features used in the literature can be found in Table 1.

[3] https://github.com/s2e-lab/BERT-Based-GitHub-Issue-Classification/

Table 7
AUC scores of the manually trained classical ML techniques on dataset D1: (a) the "Default" approach, where no class rebalancing is applied, and (b) the "Rebalanced" approach, where class rebalancing is applied.

ML technique	Approach	AUC
SGD	Default	**0.628**
	Rebalanced	0.617
NB	Default	0.608
	Rebalanced	**0.620**
LR	Default	**0.631**
	Rebalanced	0.617
SVM	Default	**0.649**
	Rebalanced	0.606
KNN	Default	**0.502**
	Rebalanced	**0.502**
RF	Default	**0.567**
	Rebalanced	0.529
DT	Default	**0.616**
	Rebalanced	0.529

Impact of class rebalancing on prediction accuracy. Dataset D1 is highly imbalanced, with only 13% of bug reports belonging to the invalid class (see details in Table 4). To investigate whether we can further improve the performance of the classical ML techniques by dealing with data imbalance, we applied the widely used Synthetic Minority Over-sampling Technique (SMOTE) (Chawla et al., 2002). SMOTE was applied after applying TF-IDF to the textual data of bug reports. Similar to previous research (Tantithamthavorn et al., 2018; Agrawal and Menzies, 2018), we fine-tuned the k parameter of SMOTE and applied it only to the training data of each fold.

Table 7 presents the results of using manually trained and optimized classical ML techniques on dataset D1. The table shows the performance of two approaches: (a) the "Default" approach, where no class rebalancing is applied, and (b) the "Rebalanced" approach, where class rebalancing is performed using SMOTE. The results indicate that both approaches yield similar performance in terms of AUC, except for DT, where AUC has decreased from 0.616 to 0.529 when using SMOTE. The small differences in the results of the two approaches could be due to randomness in data, and we do not claim their significance.

As mentioned in the above results, SMOTE does not enhance the prediction accuracy of classical ML techniques for bug report classification on the imbalanced dataset D1. This may be attributed to SMOTE's reliance on KNN, which can struggle in high-dimensional spaces, such as those encountered with textual data. An alternative potential approach to handling data imbalance involving text could be the use of generative adversarial networks (GANs) (Madani et al., 2023; Rani and Verma, 2024). For example, Rani and Verma (2024) proposed and evaluated the SMOTE Nominal Continuous-GAN (SMOTE NC-GAN) to address the issue of imbalanced text datasets. This approach combines SMOTE with GAN (comprised of two components, a generator and a discriminator). It generates synthetic data from the minority class of the dataset using SMOTE for Nominal and Continuous features (SMOTE-NC). SMOTE-NC is an extension of SMOTE that generates synthetic data for imbalanced datasets with both categorical and continuous features. The synthetically generated and the real data are fed to the discriminator for training. This allows the discriminator to improve its ability to identify and classify the two types of data accurately. In addition, the feedback loop between the discriminator and the generator, which is responsible for producing synthetic samples, enables the generator to generate more realistic and representative synthetic samples. The authors reported that empirical evaluations on various benchmark datasets demonstrated that SMOTE NC-GAN model outperforms SMOTE and its variants.

4.2. Computational cost for training the models

As shown in Table 6, the training and testing time of the manually trained and optimized classical ML techniques ranges from less than a minute to 90 min across datasets. For AutoML, we used three time slots, 30, 60, and 90 min, and we could generate models within these time slots except for D3, for which no model was generated within the 30 min slot. The training and testing time for RoBERTa is substantially high compared to AutoML and classical ML techniques. It takes 245, 171, and 299 min on D1, D2, and D3, respectively.

The memory consumption of the manually trained and optimized classical ML techniques follows the same pattern for each dataset. However, for AutoML, it increases substantially with an increment in the allocated time and the data size. The memory consumption of RoBERTa is substantially high compared to AutoML and classical ML techniques. In addition to RAM consumption, it also utilizes over 8 GB of GPU memory on D2 and D3 and 4 GB of GPU memory on D1.

Takeaways:

1. While using less memory and time, manually trained and optimized classical ML techniques, such as SGD, produce comparable results to complex techniques in classifying bug reports.
2. RoBERTa is more computationally expensive than AutoML-generated and manually trained and optimized models of classical ML techniques for bug report classification.
3. AutoML-generated models are more computationally expensive than manually optimized ones using classical ML techniques for bug report classification. However, they deliver superior performance in terms of accuracy.
4. The cost of training ML models is only a one-time investment that may not be a significant concern for practitioners when considering the adoption of ML techniques for bug report classification in practice.

4.3. Generalizability

As shown in Table 6, all of the techniques have generally performed consistently across datasets and exhibited low standard deviation. This means that the AUC for a particular technique is close to the mean AUC of each technique across all datasets except for the AutoML-30 (i.e., an AutoML-generated model with the 30 min slot). This is because no model was generated for the dataset D3 during that slot.

Takeaways:

1. AutoML, RoBERTa, and manually trained and optimized classical ML techniques perform consistently on different datasets (or projects) for classifying bug reports, i.e., with an average AUC of 0.81, 0.79, and 0.75 at best, respectively.
2. AutoML-generated models consistently outperform manually trained and optimized ML models of classical ML techniques across datasets and achieve slightly better AUC than RoBERTa. Thus, AutoML offers generalizability in classifying bug reports.

4.4. Scalability

As shown in Fig. 1, for the manually trained and optimized ML techniques, as expected, the time to obtain results generally increases as the number of bug reports increases. However, in the given context, the increment in time is small, except for the RF, which takes a relatively longer time.

For AutoML, the increment in time to obtain a model is significant when the number of bug reports increases. It is also evident in D3 (the larger dataset compared to D1 and D2) that no model was generated when using AutoML on D3 with a 30 min time slot.

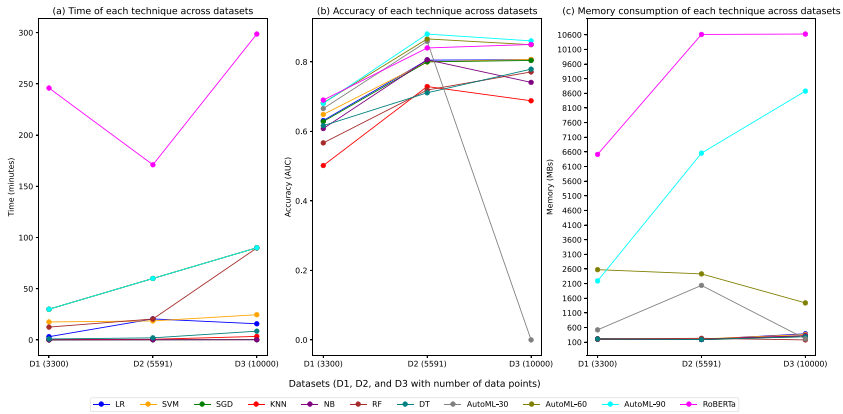

Fig. 1. Time, accuracy, and memory consumption of each technique across datasets.

Similar to AutoML, for RoBERTa, the increment in time to obtain results is significant when the number of bug reports increases. For example, for 5591 bug reports, it takes 171 min, and for 10k bug reports, it takes 298 min.

On further analysis of the scalability of AutoML and the selected classical ML techniques for bug report classification using the large dataset of bug reports (i.e., over 1.4 million bug reports, see details in Section 3.4.6), we observed the following for AutoML: no model was generated when allocating 0.5, 1, 5, 10, and 12 h. We also noticed that it is not just that the dataset is large, but it also depends on the technique. For example, we ran a few manually trained and optimized classical ML techniques (e.g., SGD) on the simple machine reported in Section 3.4.6. In addition, we monitored the RAM and CPU for the large dataset and observed that 16 GB was not enough for AutoML. However, the CPU utilization was around 60%.

The memory consumption for the manually trained and optimized classical ML techniques follows a similar trend across the datasets. However, for AutoML, memory consumption increases significantly when allocated time and the number of bug reports increases. The average RAM consumption across the three timeslots (30, 60, and 90 min) is as follows: (a) 925 MBs for 3300 bug reports, (b) 2150 MBs for 5100 bug reports, and (c) 5808 MBs for 10000 bug reports.

Similar to AutoML, the memory consumption of RoBERTa increases significantly when the number of bug reports increases. For example, for 5591 bug reports, it consumes (2513.30 RAM and 4000 GPU) and for 10k bug reports, it consumes (2116.36 and 8508 GPU).

Takeaways:

1. Increasing the size of data improves the accuracy of bug report classification. However, it also increases the time to obtain results and memory consumption.
2. Manually trained and optimized classical ML techniques (e.g., SGD) scale on large datasets, i.e., provide comparable results on large datasets compared to complex techniques in classifying bug reports, taking less time and requiring less computational power.
3. AutoML requires significantly more computational power (less than RoBERTa) and takes more time (less than RoBERTa) to produce results on large datasets than manually trained and optimized classical ML techniques. However, AutoML achieves higher accuracy in classifying bug reports.

4.5. Learning curve

To use the selected classical ML techniques and RoBERTa effectively for classifying bug reports requires knowledge of selecting an appropriate ML model, hyperparameters, and preprocessing methods. AutoML can automatically identify the most suitable ML model with suitable hyperparameters and preprocessing methods for a given dataset. For example, the AutoML tool used in this study (i.e., Auto-sklearn) can handle textual data automatically. However, similar to conventional ML, for AutoML, initial data cleaning is necessary to ensure effective results, which includes removing domain-specific stopwords.

Takeaways:

1. The effective use of conventional ML for classifying bug reports requires knowledge of selecting an appropriate ML model, hyperparameters, and preprocessing methods.
2. AutoML helps find the best model automatically with relevant hyperparameters and preprocessing methods for classifying bug reports for a given dataset.

4.6. Explainability

Global explainability (explaining the ML model). Both the manually trained and optimized and the AutoML generated models require explanation. Tools such as SHAP (Lundberg and Lee, 2017) can be used to explain the overall workings of a model. For instance, SHAP can provide information on the significance of each feature in the model. BERT-based models also require explanation. Tools such as Captum (Kokhlikyan et al., 2020) can be used to interpret a BERT model.

Furthermore, classical ML techniques such as LR, DT, SVM in linear form, and SGD trained with linear SVM or LR are generally considered more interpretable (Räz, 2024; Hastie et al., 2009) than KNN and ensemble methods like RF.

Similarly, an ensemble model generated by AutoML can be complex and difficult to understand. In addition, the process of generating an AutoML-based model is entirely black-box. However, tools such as PipelineProfiler (Ono et al., 2020) can help understand the pipeline of the ensemble model.

Local explainability (explaining the results of a prediction). RoBERTa, the selected classical ML techniques that we manually train and optimize, and AutoML do not inherently explain the prediction results for an individual instance. However, tools such as SHAP (Lundberg and Lee, 2017) and LIME (Ribeiro et al., 2016)[4] can be used for explaining the prediction results of individual instances.

Takeaway:

RoBERTa, classical ML techniques with manual training and optimization, and AutoML lack local explainability. However, classical ML techniques are generally considered more interpretable due to their simplicity.

[4] https://github.com/marcotcr/lime/issues/356

4.7. Maintenance of the model

ML models require regular updates to maintain their performance over time and adapt to changes in data. A simple retraining of a manually trained and optimized model of the classical ML techniques on recent data may not result in a new model that can handle underlying changes in the data. Since the current changes to the data may render the existing ML technique unsuitable, and an alternative technique may need to be employed for better or similar accuracy, which AutoML can support. A simple rerunning of an AutoML pipeline can find the best model to handle updates in data, as AutoML will utilize all available techniques to provide the best ensemble model. Similarly, a simple rerunning (fine-tuning) of a BERT-based model may yield a new model that can handle underlying changes in data.

Takeaway:

AutoML may be a suitable solution to streamline the maintenance of an ML model to sustain its performance over time and adapt to changes in data, i.e., AutoML can automatically find the best model to handle changes in data, utilizing all available techniques to provide the best ensemble model.

4.8. Availability of open-source tools

Several open-source tools are available for BERT, AutoML, and classical ML techniques requiring manual training and optimization. Examples of the tools for implementing the classical ML techniques evaluated in this research include sklearn (scikit-learn developers, 2024a) and Weka (Frank et al., 2016). Examples of tools for implementing AutoML include TPOT (Moore, 2024), Auto-sklearn (Feurer et al., 2015), and H20 AutoML (H2O AutoML Org, 2024). An example of a tool for implementing the state-of-the-art BERT-based models includes Transformers (Wolf et al., 2020).

4.9. Technical support

To use a particular ML library/tool, users typically look for guidance on an installation procedure, implementing techniques (e.g., creating a model and setting its parameters), and implementing a simple example. In addition, users also expect an active community (Bjarnason et al., 2023) that may address/answer any questions or issues that may arise while using the ML tool. The tools utilized in this study offer such support. For example, the official websites of Auto-sklearn, sklearn, and Transformers provide complete documentation, including an installation guide and examples for implementing techniques. In addition, they have an active community on GitHub where users can seek help and share their experiences.

Additionally, tutorial sites like Machine Learning Mastery provide multiple examples to help users better understand the relevant concepts of ML.

4.10. Trade-off analysis of the ML techniques

Table 9 shows a decision matrix that compares several ML techniques for bug report classification against the adoption concerns of practitioners.

We define an example scale based on each technique's relative advantage (plusses) and disadvantage (minuses) for each adoption attribute that practitioners consider important when selecting an ML technique. It is important to note that the defined scales for objective results of the ML techniques are only for guidance purposes, and a particular organization can adjust them accordingly. We describe our scales as follows.

- **Accuracy (AUC):** We define an example scale for accuracy (AUC) of an ML technique as follows: an AUC score between 0.75 and 0.79 is rated as +, while a score between 0.8 and 0.84 is rated as ++, and any score above 0.85 is rated as +++. Similarly, an AUC score below 0.75 is rated as −, and so on (see details in Table 8). We take the best AUC for a technique on any of the three datasets used in our study.
- **Computational cost for training the models:** The computational cost of AutoML is significantly higher than manually trained and optimized classical ML techniques. Thus, we rate AutoML as − and manually trained and optimized classical ML techniques as +. The computational cost of RoBERTa is even higher than AutoML. Thus, we rate RoBERTa as −−.
- **Generalizability:** For generalizability, we use the same scale as for the accuracy (see details in Table 8). However, the scale is applied on average AUC across the three datasets used in our study.
- **Scalability:** As our findings indicate that the manually trained and optimized classical ML techniques generally take less time to produce results, consume less memory, and can improve their results with an increment in data, except for RF, thus we rate them as +. For AutoML and RoBERTa, we observed that time and memory requirements are high. Thus, we rate them as −.
- **Learning curve:** The learning curve is more in manually trained and optimized classical ML techniques and RoBERTa. Thus, we rate AutoML as + and manually optimized classical ML techniques and RoBERTa as −.
- **Explainability:** RoBERTa, AutoML, and manually trained and optimized classical ML techniques do not inherently support explainability (i.e., local or global). In addition, the process of generating an AutoML-based model is completely black box. Thus, for global explainability, we rate AutoML and RoBERTa as − and manually trained and optimized classical ML techniques as + due to their simplicity. For local explainability, we rate all three as −.
- **Maintenance of the model:** As AutoML and RoBERTa can handle model updation in a better way than manually trained and optimized classical ML techniques, we rate AutoML and RoBERTa as + and manually trained and optimized classical ML techniques as -.
- **Availability of open-source tools:** As described in Section 4.8, several open-source tools exist for implementing RoBERTa, AutoML, and manually trained and optimized classical ML techniques. Thus, we assign the same rating to both, i.e., +.
- **Technical support:** As described in Section 4.9, adequate support is available for RoBERTa, AutoML, and manually trained and optimized classical ML techniques. Thus, we assign the same rating to three, i.e., +.

5. Discussion

In this study, we assessed several ML techniques for bug report classification against the adoption concerns of practitioners using Rana et al. (2014b)'s adoption framework for ML techniques. Our assessment combines quantitative results (obtained via experiments) with subjective arguments to evaluate the suitability of various ML techniques based on practitioners' adoption concerns. Our study provides data regarding the concerns that could be assessed agnostic of the adopting organization's context and without the involvement of practitioners. This information can now be used to compare alternatives when selecting ML techniques for a given context. This can help practitioners and researchers select the most appropriate techniques for their context. Future research may identify new concerns and provide evidence on how existing and new techniques perform regarding such concerns.

Furthermore, the proposed framework for evaluating ML techniques for bug report classification includes extensive criteria related to the characteristics of ML techniques, such as prediction accuracy, generalizability, scalability, and explainability. It is adaptable and can be readily applied or extended by others to include additional techniques or adoption factors. In addition, it can also be adapted for other tasks in software engineering, such as selecting an appropriate ML for defect prediction.

Table 8
Scale for the relative advantage (plusses) or disadvantage (minuses) for accuracy and generalizability of an ML technique.

≤ 0.54	0.55–0.59	0.60–0.64	0.65–0.69	0.70–0.74	0.75–0.79	0.80–0.84	0.85–0.89	0.90–0.94	≥ 0.95
− − − − −	− − −−	− − −	−−	−	+	++	+++	+++	+++

Table 9
Trade-off analysis of the ML techniques for bug report classification against the adoption concerns of practitioners.

Attributes	ML techniques										
	NB	LR	SVM	SGD	KNN	RF	DT	AutoML-30	AutoML-60	AutoML-90	RoBERTa
Accuracy	++	++	++	++	−	+	+	+++	+++	+++	+++
Computational cost for training the models	+	+	+	+	+	+	+	−	−	−	−−
Generalizability	−	+	+	−	− − −	−−	−	− − − − −	++	++	+
Scalability	+	+	+	+	+	−	+	−	−	−	−
Learning curve	−	−	−	−	−	−	−	+	+	+	−
Explainability (global)	+	+	+	+	+	+	+	−	−	−	−
Explainability (local)	−	−	−	−	−	−	−	−	−	−	−
Maintenance of the model	−	−	−	−	−	−	−	+	+	+	+
Availability of open-source tools	+	+	+	+	+	+	+	+	+	+	+
Technical support	+	+	+	+	+	+	+	+	+	+	+

5.1. Computational cost and scalability

Training cost: Our findings indicate that the computational cost of training the models may not be a major concern for practitioners when selecting a technique for bug report classification. Training models based on classical ML techniques, such as LR, SGD, and SVM, are relatively faster and cheaper than BERT and AutoML-based models. BERT and AutoML-based models take a considerably longer time and require more computational resources. For example, fine-tuning a pre-trained BERT-based model without GPU for bug report classification can take several hours, even on a small dataset of bug reports. However, this is a one-time investment. Thus, practitioners may prefer a model with high accuracy regardless of the required computation cost or training time.

Yet, it should be considered that every time there is concept drift (Bennin et al., 2020), the ML models need to be retrained. Retraining models from scratch continuously and frequently, such as daily or weekly, can be computationally expensive. This process can complicate parameter tuning and delay the deployment of updated ML models (Lwakatare et al., 2020). Furthermore, this will affect sustainability due to increased data centers' energy consumption and carbon emissions, which often depend on fossil fuels. In addition, increasing demand for computational power increases the usage of water for cooling purposes and speeds up hardware obsolescence, contributing to the growing issue of electronic waste.

Inference cost: The inference cost of simple models is less, similar to their training cost. The inference cost of AutoML-selected models depends on the complexity of the chosen model. For instance, this cost will be less, similar to simple models, if the chosen model is based on classical ML techniques. In contrast, the cost would be high for a complex model, such as those based on deep learning. The inference cost of a BERT-based model will be slightly higher compared to that of simple models because of the complexity of the transformer architecture. Before predicting the class of a new bug report, the bug report will go through the complete data pre-processing pipeline. The process will be costly due to the specific requirements of the transformer architecture.

Operational cost: While the training and inference cost may not be a major concern for practitioners, operational costs will still be significant. For instance, an organization will likely need GPUs or a cloud computing platform to deploy complex and resource-intensive models such as BERT-based. In the context of bug report classification, this cost may not be excessive, but it will still be significantly higher than the operational cost of classical ML models.

Scalability: As the number of bug reports or features increases, simple techniques such as RF and SVM (with non-linear kernels) will become more computationally expensive and require more time and resources. Yet, as indicated in our results, simple techniques take relatively less time and resources than AutoML and BERT-based models. In addition, some of the simple techniques, such as SGD, can scale (see Section 4.4).

AutoML will also require more time and resources when dealing with larger datasets of bug reports. While BERT-based models perform well for bug report classification, their scalability is limited when handling a large number of bug reports. Training them on a single machine can be challenging and time-consuming. Therefore, utilizing distributed training algorithms across multiple machines or modern cloud-based solutions is essential to decrease training latency and expedite convergence (Lwakatare et al., 2020). However, this will also increase computational cost.

While the cost may increase when scaling these techniques, similar to the computational cost in the context of bug report classification, this cost may not be a significant concern for practitioners when choosing a technique for bug report classification.

Even though the cost of scaling these techniques may not be a significant concern for practitioners, achieving scalability will require adopting systematic approaches when deploying trained ML models in production (Lwakatare et al., 2020). This involves carefully designing ML platforms for different end-to-end ML workflows. Such platforms often include OSS and custom-designed ecosystems for ML created using various programming languages. For example, Facebook utilizes Caffe2 (a deep learning framework) for production purposes, whereas PyTorch is used for AI research, enabling quick experimentation. Facebook follows such an approach because Caffe2 includes essential features needed for production, including performance, support for multiple platforms, and effective convergence for core ML algorithms. In contrast, PyTorch provides flexibility, debugging, and dynamic neural networks, making it ideal for quick experimentation. In addition, it provides options for inspecting and visualizing models, but this can lead to reduced performance in production.

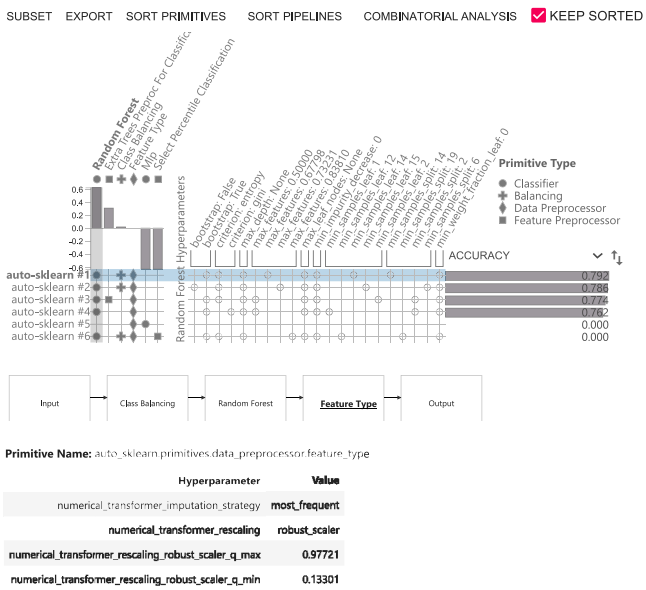

Fig. 2. An example of detailed visualization of AutoML-generated pipeline using PipelineProfiler (Ono et al., 2020).

5.2. Maintaining the performance of model

Simple models for bug report classification, such as those based on LR and SVM, may struggle to maintain their performance over time when there is concept drift (Bennin et al., 2020). In addition, they will require manual optimization, and even with that, these models may still struggle to adapt to handle concept drift (Bennin et al., 2020). The current changes to the data may render the existing ML technique unsuitable, and an alternative technique may need to be employed for better or similar performance. AutoML provides a better and more automated solution. It can automatically optimize and also provide a new model to adapt to changes in data by experimenting with various available techniques. Similarly, BERT-based models offer better solutions for maintaining their performance through fine-tuning and transfer learning.

5.3. Explainability

Simple ML techniques such as LR and SVM are inherently more explainable (Räz, 2024; Hastie et al., 2009). In contrast, ensemble models, which combine various models, and AutoML-based models, which include complex pipelines, are harder to interpret. Similarly, models based on BERT present a challenge regarding explainability due to their deep and complex architectures.

Tools, such as SHAP (Lundberg and Lee, 2017), LIME (Ribeiro et al., 2016), and Captum (Kokhlikyan et al., 2020), can be used for local (why a particular prediction was made) and global (how the model works in general) explainability. For example, SHAP has been extensively applied in various fields, including software engineering (Santos et al., 2020; Laiq et al., 2024) such as medicine, banking, power, and irrigation with promising results (Moscato et al., 2021; Zhang et al., 2022, 2020; Jamei et al., 2024).

There is also a growing trend to offer automated support for AutoML. Some existing AutoML tools, such as H2O (H2O AutoML Org, 2024) and MLJAR (Płońska and Płoński, 2021), provide such explanations automatically. To understand AutoML-generated pipelines, tools such as PipelineProfiler (Ono et al., 2020) can be used. See an example in Fig. 2 and more details in Section 5.5. Furthermore, model-agnostic methods (tools), such as SHAP (Lundberg and Lee, 2017), can be applied to models generated by AutoML. These tools can then explain the prediction of an individual instance, i.e., why a particular prediction was made. Additionally, the calculation of the importance of the features, which is used to explain the overall functioning of the model, can also be supported by these tools and automated.

BERT-based models are known for their high prediction accuracy, as shown in our results as well. However, they present a significant challenge when it comes to their explainability. They are inherently complex due to their deep architecture, which often consists of deep neural networks. The many layers and complicated connections make understanding and tracing the model's reasoning challenging. Although techniques like the attention mechanism can provide some insight into the model, they do not fully explain how it gets to its final answer (Hadi et al., 2023). In addition, as these models continue to expand and now include billions of parameters, it becomes more challenging to understand how they make decisions fully. This may not be as critical for the bug report classification context as it may be in critical contexts such as healthcare or finance. However, this needs to be addressed to improve their likelihood of adoption in practice.

5.4. Large language models for bug report classification

Large language models (LLMs) have the potential to support practitioners in bug report classification. They offer several advantages, such as high accuracy and generalizability. However, these advantages come with challenges, such as complexity and cost. In the following, we discuss the potential advantages and challenges of LLMs in the context of bug report classification.

Bug reports often contain ambiguous, complex, and context-specific text that may be challenging to handle for classical ML techniques. However, LLMs can handle it since they are pre-trained on vast corpora, including a wide range of domain-specific content, such as technical documents and programming languages. This extensive training enables LLMs to understand these nuances more effectively, making them well-suited for classifying bug reports. In addition, LLMs can generalize effectively to different contexts for bug report classification since they are trained on diverse data. LLMs can capture dependencies between words and infer missing information based on the surrounding context. Moreover, in situations where labeled data of bug reports is scarce or unavailable, LLMs that apply techniques of few-shot and zero-shot learning can be handy. Additionally, LLMs can also be effective in maintaining the performance of their model in the long term since they can adjust their weights to adopt changes in the data of bug reports.

Despite the advantages of LLMs, as mentioned above, they are not without challenges. For example, the computational cost of LLMs (further discussed in Section 5.1) is considerably higher than that of simple ML techniques. Similarly, they are complex and lack explainability (further discussed in Section 5.3). To understand and keep them running, an adopting organization will likely need experts. Despite their pre-training on vast corpora, they may still lack domain-specific information. Moreover, implementing LLM-based solutions will require a suitable infrastructure, which will be costly. Without adequate infrastructure, the entire project may be at risk of failure.

Another significant concern related to the adoption of LLMs is data privacy and the challenge of assessing the reliability and transparency of these models. For example, in the case of commercial models such as GPT-4 from OpenAI, there is limited knowledge of how the model is trained and updated. As these models continuously evolve, tracking history or understanding how they may handle sensitive or proprietary information becomes complex. In addition, these models are beyond the control of an organization, further raising reliability concerns.

Consequently, the decision to adopt LLMs for bug report classification must consider the balance between their capabilities and limitations.

5.5. AutoML for bug report classification

Our findings indicate that AutoML can overcome several shortcomings related to the adoption readiness of conventional ML for bug report classification:

- AutoML automatically selects a suitable ML model with appropriate hyperparameters and data preprocessing methods for a given dataset.
- AutoML improves performance over manually trained and optimized ML models of classical ML techniques and can achieve slightly better performance than RoBERTa.
- AutoML can perform consistently with higher accuracy on different datasets (projects), thus offering generalizability.
- AutoML helps in maintaining an ML model to sustain its accuracy over time and adapt to changes in data.

Besides the advantages described above, AutoML also poses some challenges for bug report classification. For example, the computational cost of AutoML is higher than the one for manually trained and optimized classical ML techniques. Furthermore, the process of generating a model using AutoML is a black box, which impacts general explainability (see Section 4.6), which is a concern often raised by practitioners (Paleyes et al., 2022; Bhatt et al., 2020). However, tools such as PipelineProfiler (Ono et al., 2020) can be used to interactively visualize and explore AutoML-generated pipelines to get more insight into the black box. In Fig. 2, we show an example of how the PipelineProfiler can be used to explore a pipeline generated by Auto-sklearn.

The figure explains the model auto-sklearn#1 in the highlighted row. It shows (i) the ML technique used for the generated model, i.e., Random Forest (RF), (ii) the hyperparameters of RF, (iii) the used data and feature processing methods, (iv) the applied class balancing approach, and (v) the accuracy of the generated model. In addition, at the bottom, the figure shows a complete flow of the pipeline, starting from input to output.

Although tools are available (as mentioned above and in Section 5.3) to help understand AutoML-generated models, the lack of transparency in AutoML decision-making processes remains a significant challenge. This issue impacts both understanding and trust in the models produced (Azevedo et al., 2024). As highlighted by Krauß et al. (2020), AutoML may include sophisticated approaches for the automatic selection of techniques, hyperparameter optimization, and data processing. However, it lacks support for a thorough diagnosis of models, further reducing the interpretability and trust in the models generated by AutoML.

Another challenge of AutoML associated with its adoption in practice is the lack of support for deploying models. Barbudo et al. (2023) reported that AutoML does not yet cover the automation of the whole process, which affects the practicality of the proposals when adopting them in practice. Alamin and Uddin (2022) reported that around 13% of AutoML practitioners experience difficulties with model loading and deployment. This is critical since AutoML is expected to streamline the model deployment process. Similarly, Xin et al. (2021) reported that currently, AutoML is mainly used to automate model training and requires users to handle data preprocessing and postprocessing with other tools. This contradicts the advantages claimed by AutoML tool providers.

Furthermore, implementing AutoML-based solutions requires a suitable infrastructure, which often involves considerable manual effort. Without adequate infrastructure, the entire project may be at risk of failure. In addition, infrastructure challenges can significantly impact the project timeline, as they often involve long-term investment decisions, especially within larger companies (Baier et al., 2019).

Beyond this, another important issue, as raised by Polzer and Thalmann (2022) that needs to be considered, is the upcoming AI regulations worldwide. AutoML not only aims to automate repetitive and time-consuming tasks in ML development but also democratize AI, enabling non-experts to automatically develop AI-based systems without data scientists for their use in practice. This raises questions about how AutoML will fit into these regulatory frameworks and what requirements might be imposed on AI systems developed using AutoML. The regulations apply not only to the resulting AI-based systems but also to the business systems in which they will be integrated, necessitating that they meet the requirement of human oversight (Polzer and Thalmann, 2022).

5.6. Simple ML techniques for bug report classification

Our results indicate that classical ML techniques are also effective in classifying bug reports, performing comparably to complex techniques such as BERT-based and deep learning approaches. Additionally, classical techniques like SGD have a lower computational cost than AutoML, BERT-based, and deep learning-based techniques. As shown in Table 6 and Fig. 1, classical techniques such as SGD can scale on large datasets for bug report classification. Hence, classical ML techniques could be a viable and sustainable solution for bug report classification due to their simplicity, accuracy, and cost-effectiveness.

6. Validity threats

The section describes the potential validity threats of our study.

Generalizability of results. To improve the generalizability of the results, more than one dataset and data from both open-source and closed-source contexts were used in the study.

Our results would be interesting for large-scale software development organizations. We also provide an adaptable framework that others can use to find a suitable technique for their context. Moreover, despite the variations in context, size, and class distribution of the used datasets, the models performed reasonably across all three datasets. Our evaluation results also align with those reported in the literature. Additionally, the two techniques we applied, AutoML and RoBERTa, are designed to generate models tailored to specific datasets. AutoML will create a model using all available techniques for a new dataset. Similarly, the fine-tuned model based on RoBERTa will adjust its weights according to a new dataset. This is evident in both our results and the literature. This gives us confidence that these techniques will perform similarly on datasets from other contexts.

In this work, we perform binary classification of bug reports. However, the framework developed in this study is adaptable and can be used to assess the performance of the proposed techniques to classify bug reports into more than two categories. Furthermore, since the task is the same, we contend that our results can be applicable/relevant for contexts where bug reports are classified into more than two categories, provided sufficient ground truth is available for each label for training a model.

Choice of data preprocessing approaches. Different data preprocessing approaches can be used to prepare data for training ML models. When comparing the performance of different ML techniques, the choice of an inappropriate data preprocessing approach may introduce bias. To avoid such bias, we chose data preprocessing approaches that were applicable and comparable for the three techniques. With the classical ML techniques (e.g., LR and SVM), we use the traditional approach of TF-IDF. For the BERT-based model, we use a BERT-based tokenizer; for AutoML, we let AutoML decide which approach to use.

This study uses TF-IDF as a data preprocessing approach with classical ML techniques, as it aligns with the study's objectives (as mentioned above). However, other approaches, such as word2vec and BERT-based embeddings, can also be used with classical ML techniques.

Validity of the evaluation approach. Different evaluation approaches, such as 10-fold cross-validation and testing on a fixed set, can be used to evaluate the performance of ML techniques. To reduce the potential bias of an evaluation approach and to improve the reliability of results, similar to previous work (see Section 2.2), we used a 10-fold cross-validation approach (for details, see Section 3.4.2). This approach reduces the experimental bias (Witten and Frank, 2002) and improves the results' reliability.

Validity of the evaluation metrics. Herzig et al.'s (Herzig et al., 2013) dataset and the dataset from CSS are imbalanced (see Section 3.3). To reduce potential bias in interpreting the results and improve the results' reliability, we used a metric considered robust to data imbalance, i.e., AUC (Huang and Ling, 2005).

Reliability of training data. The data used for training the ML techniques could be a source of validity threats. For example, bug reports could have been misclassified. To reduce this threat, we utilized two widely used datasets for bug report classification provided by Herzig et al. (2013) and Kallis et al. (2023a), see Section 2.2. The dataset by Herzig et al. (2013) has also been manually validated. Additionally, we used a dataset from CSS that includes bug reports with finished status, i.e., classified as valid or invalid, representing judgments of more than one domain expert.

Relevance of technology adoption concerns. To mitigate the threat of using an inadequate adoption framework, we adapted the existing framework by Rana et al. (2014b) that was specifically developed for the adoption of ML-based solutions. It has also been used previously in the context of defect prediction (Rana et al., 2014a).

7. Conclusion and future work

In this study, we assessed several ML techniques for bug report classification against the adoption concerns of practitioners. Our assessment includes a decision matrix that combines quantitative results with subjective arguments to evaluate the suitability of ML techniques. The matrix and prioritization of concerns can assist both practitioners and researchers in selecting the most appropriate ML techniques for their organizational context.

Our study's contribution helps to visualize the trade-off between several ML techniques for bug report classification against the adoption concerns of practitioners. For instance, it shows that AutoML can overcome several shortcomings related to the adoption readiness of conventional ML for bug report classification. AutoML helps in selecting an appropriate ML model with suitable hyperparameters and data preprocessing methods automatically for a given dataset. AutoML-generated models perform better than manually trained and optimized ML models and consistently deliver higher accuracy across different datasets, thus offering generalizability. AutoML also supports updating the ML model to maintain its performance over time and adapt to changes in data. However, like conventional ML, AutoML lacks explainability, and the process of generating a model using AutoML is a black box.

Using the evaluation framework and the approach to visualize tradeoffs, as presented in the paper, supports making informed decisions when choosing ML techniques. Since the overall approach is not locked to the specific techniques, datasets, or factors we have selected here, others could easily use and adapt it for additional techniques or concerns.

In the future, we aim to use (and improve) the decision matrix in real industrial settings when practitioners are selecting ML techniques for bug report classification.

CRediT authorship contribution statement

Muhammad Laiq: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Nauman bin Ali:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Jürgen Börstler:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Conceptualization. **Emelie Engström:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been supported by ELLIIT; the Strategic Research Area within IT and Mobile Communications, funded by the Swedish Government. The work has also been supported by a research grant for the GIST project (reference number 20220235) from the Knowledge Foundation in Sweden.

Data availability

The code and information about the open source data used in this study can be found at: https://github.com/laiqujan/ML-techniques-for-bug-report-classification/.

References

Agrawal, A., Menzies, T., 2018. Is "better data" better than "better data miners"? on the benefits of tuning SMOTE for defect prediction. In: Proceedings of the 40th International Conference on Software Engineering. pp. 1050–1061.

Aktas, E.U., Yilmaz, C., 2020. Automated issue assignment: results and insights from an industrial case. Empir. Softw. Eng. 25 (5), 3544–3589.

Alamin, M.A.A., Uddin, G., 2022. Challenges and barriers of using low code software for machine learning. arXiv preprint arXiv:2211.04661.

Ali, N.B., 2016. Is effectiveness sufficient to choose an intervention?: Considering resource use in empirical software engineering. In: Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement. pp. 54:1–54:6.

Alpaydin, E., 2020. Introduction to Machine Learning. MIT Press.

Antoniol, G., Ayari, K., Di Penta, M., Khomh, F., Guéhéneuc, Y.-G., 2008. Is it a bug or an enhancement? A text-based approach to classify change requests. In: Proceedings of the Conference of the Center for Advanced Studies on Collaborative Research: Meeting of Minds. pp. 304–318.

Azevedo, K., Quaranta, L., Calefato, F., Kalinowski, M., 2024. A multivocal literature review on the benefits and limitations of automated machine learning tools. arXiv preprint arXiv:2401.11366.

Bachmann, A., Bernstein, A., 2009. Software process data quality and characteristics: a historical view on open and closed source projects. In: Mens, T., Mens, K., Wermelinger, M. (Eds.), Proceedings of the Joint International and Annual ERCIM Workshops on Principles of Software Evolution (IWPSE) and Software Evolution (Evol) Workshops, Amsterdam, Netherlands, August 24-28, 2009. ACM, pp. 119–128.

Baier, L., Jöhren, F., Seebacher, S., 2019. Challenges in the deployment and operation of machine learning in practice. In: ECIS, vol. 1.

Barbudo, R., Ventura, S., Romero, J.R., 2023. Eight years of AutoML: categorisation, review and trends. Knowl. Inf. Syst. 65 (12), 5097–5149.

Bennin, K.E., Ali, N.B., Börstler, J., Yu, X., 2020. Revisiting the impact of concept drift on just-in-time quality assurance. In: Proceedings of the 20th IEEE International Conference on Software Quality, Reliability and Security, QRS 2020, Macau, China, December 11-14, 2020. IEEE, pp. 53–59.

Bharadwaj, S., Kadam, T., 2022. Github issue classification using bert-style models. In: Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering. pp. 40–43.

Bhatt, U., Xiang, A., Sharma, S., Weller, A., Taly, A., Jia, Y., Ghosh, J., Puri, R., Moura, J.M., Eckersley, P., 2020. Explainable machine learning in deployment. In: Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency. pp. 648–657.

Bjarnason, E., Åberg, P., Ali, N.B., 2023. Software selection in large-scale software engineering: A model and criteria based on interactive rapid reviews. Empir. Softw. Eng. 28 (2), 51.

Borg, M., Jonsson, L., Engström, E., Bartalos, B., Szabó, A., 2022. Adopting automated bug assignment in practice: A longitudinal case study at ericsson. arXiv preprint arXiv:2209.08955.

Calefato, F., Quaranta, L., Lanubile, F., Kalinowski, M., 2023. Assessing the use of automl for data-driven software engineering. arXiv preprint arXiv:2307.10774.

Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P., 2002. SMOTE: synthetic minority over-sampling technique. J. Artificial Intelligence Res. 16, 321–357.

Chawla, I., Singh, S.K., 2015. An automated approach for bug categorization using fuzzy logic. In: Proceedings of the 8th India Software Engineering Conference. pp. 90–99.

Colavito, G., Lanubile, F., Novielli, N., 2022. Issue report classification using pre-trained language models. In: Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering. pp. 29–32.

Colavito, G., Lanubile, F., Novielli, N., 2023. Few-shot learning for issue report classification. In: Proceedings of the 2nd International Workshop on Natural Language-Based Software Engineering. pp. 16–19.

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

Du, X., Liu, Z., Li, C., Ma, X., Li, Y., Wang, X., 2024. LLM-BRC: A large language model-based bug report classification framework. Softw. Qual. J. 1–21.

Fan, Y., Xia, X., Lo, D., Hassan, A.E., 2018. Chaff from the wheat: Characterizing and determining valid bug reports. IEEE Trans. Softw. Eng. 46 (5), 495–525.

Feurer, M., Klein, A., Eggensperger, K., Springenberg, J., Blum, M., Hutter, F., 2015. Efficient and robust automated machine learning. Adv. Neural Inf. Process. Syst. 28.

Frank, E., Hall, M.A., Witten, I.H., 2016. The WEKA Workbench. Morgan Kaufmann.

H2O AutoML Org, 2024. H2O AutoML. https://docs.h2o.ai/h2o/latest-stable/h2o-docs/automl.html. (Online; Accessed 01 July 2024).

Hadi, M.U., Qureshi, R., Shah, A., Irfan, M., Zafar, A., Shaikh, M.B., Akhtar, N., Wu, J., Mirjalili, S., et al., 2023. A survey on large language models: Applications, challenges, limitations, and practical usage. Authorea Preprints.

Hastie, T., Tibshirani, R., Friedman, J.H., Friedman, J.H., 2009. The Elements of Statistical Learning: Data Mining, Inference, and Prediction, second ed. Springer.

He, J., Xu, L., Fan, Y., Xu, Z., Yan, M., Lei, Y., 2020. Deep learning based valid bug reports determination and explanation. In: Proceedings of the 31st International Symposium on Software Reliability Engineering. pp. 184–194.

Herbold, S., Trautsch, A., Trautsch, F., 2020. On the feasibility of automated prediction of bug and non-bug issues. Empir. Softw. Eng. 25, 5333–5369.

Herzig, K., Just, S., Zeller, A., 2013. It's not a bug, it's a feature: how misclassification impacts bug prediction. In: Proceedings of the 35th International Conference on Software Engineering. pp. 392–401.

Huang, J., Ling, C.X., 2005. Using AUC and accuracy in evaluating learning algorithms. IEEE Trans. Knowl. Data Eng. 17 (3), 299–310.

Hutter, F., Kotthoff, L., Vanschoren, J., 2019. Automated Machine Learning: Methods, Systems, Challenges. Springer Nature.

Izadi, M., 2022. CatIss: An intelligent tool for categorizing issues reports using transformers. In: Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering. pp. 44–47.

Jamei, M., Ali, M., Karbasi, M., Karimi, B., Jahannemaei, N., Farooque, A.A., Yaseen, Z.M., 2024. Monthly sodium adsorption ratio forecasting in rivers using a dual interpretable glass-box complementary intelligent system: Hybridization of ensemble TVF-EMD-VMD, Boruta-SHAP, and explainable GPR. Expert Syst. Appl. 237, 121512.

Joulin, A., Grave, E., Bojanowski, P., Mikolov, T., 2016. Bag of tricks for efficient text classification. arXiv preprint arXiv:1607.01759.

Kallis, R., Chaparro, O., Di Sorbo, A., Panichella, S., 2022. Nlbse'22 tool competition. In: Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering. pp. 25–28.

Kallis, R., Di Sorbo, A., Canfora, G., Panichella, S., 2019. Ticket tagger: Machine learning driven issue classification. In: Proceedings of the 35th International Conference on Software Maintenance and Evolution. pp. 406–409.

Kallis, R., Izadi, M., Pascarella, L., Chaparro, O., Rani, P., 2023a. The NLBSE'23 tool competition. In: Proceedings of the 2nd International Workshop on Natural Language-Based Software Engineering.

Kallis, R., Izadi, M., Pascarella, L., Chaparro, O., Rani, P., 2023b. The NLBSE'23 tool competition. In: Proceedings of the 2nd International Workshop on Natural Language-Based Software Engineering. pp. 1–8.

Kokhlikyan, N., Miglani, V., Martin, M., Wang, E., Alsallakh, B., Reynolds, J., Melnikov, A., Kliushkina, N., Araya, C., Yan, S., et al., 2020. Captum: A unified and generic model interpretability library for pytorch. arXiv preprint arXiv:2009.07896.

Krauß, J., Pacheco, B.M., Zang, H.M., Schmitt, R.H., 2020. Automated machine learning for predictive quality in production. Procedia CIRP 93, 443–448.

Laiq, M., 2023. An intelligent tool for classifying issue reports. In: Proceedings of the 2nd International Workshop on Natural Language-Based Software Engineering. pp. 13–15.

Laiq, M., Ali, N.b., Börstler, J., Engström, E., 2024. Industrial adoption of machine learning techniques for early identification of invalid bug reports. Empir. Softw. Eng. 29 (5), 130.

Laiq, M., Ali, N.b., Böstler, J., Engström, E., 2022. Early identification of invalid bug reports in industrial settings–a case study. In: Proceedings of the 23rd International Conference on Product-Focused Software Process Improvement. pp. 497–507.

Li, Z., Pan, M., Pei, Y., Zhang, T., Wang, L., Li, X., 2022. DeepLabel: Automated issue classification for issue tracking systems. In: Proceedings of the 13th Asia-Pacific Symposium on Internetware. pp. 231–241.

Lundberg, S.M., Lee, S.-I., 2017. A unified approach to interpreting model predictions. Adv. Neural Inf. Process. Syst. 30.

Lwakatare, L.E., Raj, A., Crnkovic, I., Bosch, J., Olsson, H.H., 2020. Large-scale machine learning systems in real-world industrial settings: A review of challenges and solutions. Inf. Softw. Technol. 127, 106368.

Madani, M., Motameni, H., Mohamadi, H., 2023. KNNGAN: an oversampling technique for textual imbalanced datasets. J. Supercomput. 79 (5), 5291–5326.

Molnar, C., 2018. A guide for making black box models explainable. URL: https://christophm.github.io/interpretable-ml-book2, 3.

Moore, J.H., 2024. TPOT AutoML. https://automl.info/tpot/. (Online; accessed 01 July 2024).

Moscato, V., Picariello, A., Sperlí, G., 2021. A benchmark of machine learning approaches for credit score prediction. Expert Syst. Appl. 165, 113986.

Oliveira, P., Andrade, R.M., Barreto, I., Nogueira, T.P., Bueno, L.M., 2021. Issue auto-assignment in software projects with machine learning techniques. In: Proceedings of the 8th International Workshop on Software Engineering Research and Industrial Practice. pp. 65–72.

Ono, J.P., Castelo, S., Lopez, R., Bertini, E., Freire, J., Silva, C., 2020. Pipelineprofiler: A visual analytics tool for the exploration of automl pipelines. IEEE Trans. Vis. Comput. Graphics 27 (2), 390–400.

Otoom, A.F., Al-jdaeh, S., Hammad, M., 2019. Automated classification of software bug reports. In: Proceedings of the 9th International Conference on Information Communication and Management. pp. 17–21.

Paleyes, A., Urma, R.-G., Lawrence, N.D., 2022. Challenges in deploying machine learning: A survey of case studies. ACM Comput. Surv. 55 (6), 1–29.

Pandey, N., Sanyal, D.K., Hudait, A., Sen, A., 2017. Automated classification of software issue reports using machine learning techniques: an empirical study. Innov. Syst. Softw. Eng. 13, 279–297.

Pingclasai, N., Hata, H., Matsumoto, K.-i., 2013. Classifying bug reports to bugs and other requests using topic modeling. In: Proceedings of the 20th Asia-Pacific Software Engineering Conference, vol. 2, pp. 13–18.

Płońska, A., Płoński, P., 2021. MLJAR: State-of-the-art automated machine learning framework for tabular data. Version 0.10.3.

Polzer, A.K., Thalmann, S., 2022. The impact of AutoML on the AI development process.

Qin, H., Sun, X., 2018. Classifying bug reports into bugs and non-bugs using LSTM. In: Proceedings of the 10th Asia-Pacific Symposium on Internetware. pp. 1–4.

Rana, R., Staron, M., Berger, C., Hansson, J., Nilsson, M., Meding, W., 2014a. The adoption of machine learning techniques for software defect prediction: An initial industrial validation. In: Proceedings of the Joint Conference on Knowledge-Based Software Engineering. pp. 270–285.

Rana, R., Staron, M., Hansson, J., Nilsson, M., Meding, W., 2014b. A framework for adoption of machine learning in industry for software defect prediction. In: Proceedings of the 9th International Conference on Software Engineering and Applications. pp. 383–392.

Rani, P., Verma, O.P., 2024. Modified-generative adversarial networks for imbalance text classification. Multimedia Tools Appl. 1–20.

Räz, T., 2024. ML interpretability: Simple isn't easy. Stud. Hist. Phil. Sci. 103, 159–167.

Ribeiro, M.T., Singh, S., Guestrin, C., 2016. "Why should I trust you?" Explaining the predictions of any classifier. In: Proceedings of the 22nd International Conference on Knowledge Discovery and Data Mining. pp. 1135–1144.

Santos, G., Figueiredo, E., Veloso, A., Viggiato, M., Ziviani, N., 2020. Predicting software defects with explainable machine learning. In: Proceedings of the XIX Brazilian Symposium on Software Quality. pp. 1–10.

scikit-learn developers, 2024. TfidfVectorizer. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html. (Online; Accessed 01 July 2024).

scikit-learn developers, 2024a. Scikit-learn machine learning in Python. https://scikit-learn.org/stable/index.html. (Online; Accessed 01 July 2024).

scikit-learn developers, 2024b. Sklearn metrics. https://scikit-learn.org/stable/api/sklearn.metrics.html. (Online; Accessed 01 July 2024).

Siddiq, M.L., Santos, J.C., 2022. Bert-based github issue report classification. In: Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering. pp. 33–36.

Tanaka, K., Monden, A., Yücel, Z., 2019. Prediction of software defects using automated machine learning. In: Proceedings of the 20th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing. pp. 490–494.

Tantithamthavorn, C., Hassan, A.E., Matsumoto, K., 2018. The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. IEEE Trans. Softw. Eng. 46 (11), 1200–1219.

Terdchanakul, P., Hata, H., Phannachitta, P., Matsumoto, K., 2017. Bug or not? bug report classification using n-gram idf. In: Proceedings of the 33rd International Conference on Software Maintenance and Evolution. pp. 534–538.

Trautsch, A., Herbold, S., 2022. Predicting issue types with sebert. In: Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering. pp. 37–39.

Ř.ehůřek, R., Sojka, P., 2010. Software Framework for Topic Modelling with Large Corpora. In: Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks. pp. 45–50.

Witten, I.H., Frank, E., 2002. Data mining: practical machine learning tools and techniques with Java implementations. ACM Sigmod Rec. 31 (1), 76–77.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., 2012. Experimentation in Software Engineering.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al., 2020. Transformers: State-of-the-art natural language processing. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing: System Demonstrations. pp. 38–45.

Xin, D., Wu, E.Y., Lee, D.J.-L., Salehi, N., Parameswaran, A., 2021. Whither automl? Understanding the role of automation in machine learning workflows. In: Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems. pp. 1–16.

Zanetti, M.S., Scholtes, I., Tessone, C.J., Schweitzer, F., 2013. Categorizing bugs with social networks: a case study on four open source software communities. In: Proceedings of the 35th International Conference on Software Engineering. pp. 1032–1041.

Zhang, D., Tsai, J.J., 2003. Machine learning and software engineering. Softw. Qual. J. 11, 87–119.

Zhang, Y., Weng, Y., Lund, J., 2022. Applications of explainable artificial intelligence in diagnosis and surgery. Diagnostics 12 (2), 237.

Zhang, K., Xu, P., Zhang, J., 2020. Explainable AI in deep reinforcement learning models: A shap method applied in power system emergency control. In: 2020 IEEE 4th Conference on Energy Internet and Energy System Integration. EI2, IEEE, pp. 711–716.

Zhou, Y., Tong, Y., Gu, R., Gall, H., 2016. Combining text mining and data mining for bug report classification. J. Softw.: Evol. Process. 28 (3), 150–176.

Zolkeply, M.S., Shao, J., 2019. Classifying software issue reports through association mining. In: Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing. pp. 1860–1863.